E-COMMERCE SALES ANALYTICS USING DUCKDB AND RABBITMQ

A PRACTICAL CASE STUDY REPORT

Submitted as a practical case study in partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

ARTIFICIAL INTELLIGENCE (AI) & DATA SCIENCE

Submitted by

Name of Student	Roll Number
Harsh Birje	225A8127
Aayush Chavanke	225A8128
Aniket Nikam	225A8135
Lokesh Ture	225A8138

Under the guidance of

Prof. Akshay Jadhav



Department of Artificial Intelligence (AI) & Data Science

SIES GRADUATE SCHOOL OF TECHNOLOGY (AUTONOMOUS)

Affiliated to University of Mumbai

Nerul, Navi Mumbai – 400706

A.Y. 2026–27

CERTIFICATE

This is to certify that the practical case study report entitled

"E-Commerce Sales Analytics using DuckDB and RabbitMQ"

is a bona fide work carried out by the following students of the Department of Artificial Intelligence (AI) & Data Science, SIES Graduate School of Technology (Autonomous), in partial fulfillment of the requirements for the award of the Bachelor of Engineering degree of the University of Mumbai during the academic year 2026–27.

Student	**Roll Number**
Harsh Birje	225A8127
Aayush Chavanke	225A8128
Aniket Nikam	225A8135
Lokesh Ture	225A8138

Prof. Akshay Jadhav	**Head of Department**
Faculty Guide	Artificial Intelligence (AI) & Data Science

Date: _____ Place: Nerul, Navi Mumbai

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty guide, **Prof. Akshay Jadhav**, for his guidance, technical suggestions, and regular reviews throughout this practical case study. His feedback helped us connect the theoretical ideas of data cleaning, ETL, dimensional modeling, OLAP, columnar storage, and event-driven ingestion with a working and reproducible implementation.

We thank the Department of Artificial Intelligence (AI) & Data Science, SIES Graduate School of Technology (Autonomous), for providing the academic environment and resources required to complete the work. We are also grateful to the Head of Department, faculty members, laboratory staff, and our classmates for their constructive suggestions.

Finally, we acknowledge the maintainers of DuckDB, RabbitMQ, FastAPI, Streamlit, Apache Parquet, Docker, Kubernetes, pandas, and the public technical documentation referenced in this report. The implementation, measurements, diagrams, and interpretations presented here were prepared for this practical case study.

Case Study Team

Harsh Birje

Aayush Chavanke

Aniket Nikam

Lokesh Ture

ABSTRACT

E-commerce data arrives from files, application programming interfaces, and live order events. Analysts need reliable measures of revenue, growth, customer retention, product performance, regional demand, and returns; however, repeated analysis directly on raw comma-separated value files is slow and vulnerable to inconsistent cleaning rules. This practical case study develops a compact data engineering platform that converts raw transactional data into trustworthy analytical information.

The batch pipeline processes 250,250 raw order-line records. It preserves the source in a raw landing table, removes duplicate line identifiers, standardizes categorical values, safely converts data types, and applies business rules to quantity, price, discount, dates, and identifiers. Invalid records are not silently discarded: 1,445 rows are retained with rejection reasons. The accepted 248,555 rows are organized into date, customer, and product dimensions with a central sales fact table.

Reusable SQL views calculate daily revenue, monthly growth, and the percentage of repeat customers. Window functions, `GROUPING SETS`, `ROLLUP`, and `CUBE` support decision-level analysis. Clean data is exported to year/month partitioned ZSTD-compressed Parquet. A reproducible benchmark compares CSV, Parquet, and native DuckDB scans under one-thread and four-thread settings.

A Streamlit dashboard communicates the analytical results. A separate near-real-time path accepts validated JSON through FastAPI, publishes persistent messages to a durable RabbitMQ queue, consumes them with manual acknowledgements, inserts events idempotently into DuckDB, and refreshes live metrics. Docker Compose and Kubernetes manifests document deployment. The final platform covers ingestion, data quality, ETL, warehouse design, OLAP, optimization, visualization, streaming, testing, and deployment in one explainable practical case study.

Keywords: Data Engineering, DuckDB, RabbitMQ, ETL, OLAP, Parquet, Streamlit, FastAPI, Data Warehouse, Near-Real-Time Pipeline.

Contents

Certificate i

Acknowledgement ii

Abstract iii

List of Figures vii

List of Tables viii

List of Abbreviations ix

1 INTRODUCTION 1

1.1 Practical Case Study Background 1

1.1.1 Practical Case Study Theme 1

1.2 Motivation and Need 2

1.2.1 Why a Warehouse Model? 2

1.2.2 Why RabbitMQ? 2

1.3 Problem Statement and Objectives 2

1.3.1 Problem Statement 2

1.3.2 Objectives 3

1.4 Scope and Report Organization 3

2 LITERATURE SURVEY 5

2.1 Analytical Data Management 5

2.1.1 CSV and Columnar Storage 5

2.1.2 Python and SQL Responsibilities 5

2.2 Dimensional Modeling and OLAP 5

2.2.1 Reusable Business Metrics 6

2.2.2 Window and Multidimensional Functions 6

2.3 Event-Driven Ingestion 6

2.3.1 Message Reliability 6

2.4 Comparison of Approaches 7

3 PROPOSED SYSTEM 8

3.1 System Architecture 8

3.1.1 Architectural Principles 8

3.2 System Components 9

3.3 Functional and Non-functional Requirements 10

3.4 Batch and Streaming Flow 10

3.5 Technology Selection 11

4 DESIGN AND METHODOLOGY 12

4.1 Data Source and Exploration 12

4.2 ETL and Data Quality Rules 13

4.3 Relational and Dimensional Model 14

4.4 Reusable SQL Views and Window Functions 16

4.5 OLAP and Query Optimization 16

4.6 RabbitMQ Live Pipeline 17

5 RESULTS AND DISCUSSION 18

5.1 Data Quality Results 18

5.2 Business KPIs 19

5.3 Category, Region, and Time Analysis 20

5.4 Performance Evaluation 21

5.5 Dashboard and Business Storytelling 22

5.6 Live Pipeline Validation 23

5.7 Requirement Traceability and Bonus Work 24

6 CONCLUSION AND FUTURE SCOPE 25

6.1 Conclusion 25

6.2 Limitations 25

6.3 Future Scope 26

6.4 Learning Outcomes 26

Bibliography 27

A EXECUTION GUIDE **28**

A.1 One-Click Windows Start 28

A.2 Batch Commands 28

A.3 Full RabbitMQ Stack 28

A.4 Suggested Demonstration Sequence 28

B TEAM CONTRIBUTION **29**

List of Figures

3.1 Batch analytical platform 8
3.2 Near-real-time RabbitMQ order path 11

4.1 ETL quality gates and rejected-record side output 14
4.2 Analytical star schema and quality quarantine 15

5.1 Realized revenue by product category 20
5.2 Regional and time-based analysis 20
5.3 Median query time by storage and thread mode 21
5.4 Streamlit analytical dashboard 22

List of Tables

1.1 Practical case study objectives and implementation evidence 3

2.1 Comparison of analytical and ingestion approaches 7

3.1 System components and responsibilities 9
3.2 System requirements and acceptance evidence 10

4.1 Source profile 13
4.2 Cleaning and validation rules 14
4.3 Warehouse tables and grain 15

5.1 Data-quality and storage results 18
5.2 Business KPI summary 19
5.3 Storage-format and thread benchmark 21
5.4 RabbitMQ live-path validation 23
5.5 Requirement-to-evidence traceability 24

B.1 Primary responsibilities of team members 29

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
CLI	Command Line Interface
CSV	Comma-Separated Values
DBMS	Database Management System
ETL	Extract, Transform, and Load
FK	Foreign Key
HPA	Horizontal Pod Autoscaler
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
OLAP	Online Analytical Processing
PK	Primary Key
PVC	Persistent Volume Claim
REST	Representational State Transfer
SQL	Structured Query Language
ZSTD	Zstandard compression algorithm

CHAPTER 1

INTRODUCTION

1.1 Practical Case Study Background

Modern e-commerce applications continuously create operational records such as orders, line items, customers, payments, returns, and delivery updates. These records are suitable for transaction processing, but management questions need a different structure. Decision makers ask how revenue changes over time, which categories or regions perform best, whether customers return, and where quality losses occur. Answering these questions consistently requires a data engineering process rather than an isolated charting script.

The practical case study implements a compact analytical platform around DuckDB. DuckDB is an embedded analytical database that executes vectorized SQL inside the application process. It can read CSV and Parquet directly, create persistent tables and indexes, execute joins, window functions, and multidimensional aggregations, and expose results to Python. These properties make it suitable for a laboratory implementation that demonstrates warehouse concepts without requiring a large distributed cluster.

Two ingestion time scales are combined. The batch path loads historical CSV data and a captured public API snapshot. The event path accepts a new order as JSON and transports it through RabbitMQ before storage. Both paths ultimately provide metrics to a Streamlit interface. The practical case study therefore demonstrates periodic historical processing and continuously arriving events within the same business domain.

1.1.1 Practical Case Study Theme

The selected practical case study is *E-Commerce Sales Analytics using DuckDB*. The implementation extends the basic requirement by adding deterministic source generation, public API capture, data-quality quarantine, dimensional modeling, reusable SQL views, Parquet optimization, benchmarks, a dashboard, REST ingestion, RabbitMQ messaging, container configuration, Kubernetes manifests, tests, and reproducible evidence files.

1.2 Motivation and Need

Raw CSV is portable and easy to inspect, but it does not enforce types, relationships, or business rules. A date may contain text, a quantity may be negative, and category names may vary only in capitalization. If each analyst cleans these problems independently inside a chart, the resulting metrics may contradict one another. A controlled pipeline centralizes the rules and records which rows were accepted or rejected.

Repeated full scans of row-oriented text also add parsing cost. Analytical queries normally read a subset of columns and aggregate many rows. Columnar Parquet and persistent DuckDB tables reduce repeated parsing and support projection and filter pushdown. The case study measures this effect rather than assuming it.

1.2.1 Why a Warehouse Model?

The raw source is a wide transaction file. The warehouse separates descriptive entities into dimensions and measurable events into a fact table. This design produces stable customer, product, and date keys, reduces repeated descriptive values, and makes business measures reusable. The model is deliberately simple enough that its grain and join paths can be explained during the practical demonstration.

1.2.2 Why RabbitMQ?

A direct API-to-database write tightly couples request capture to storage availability. RabbitMQ places a durable buffer between the API and the consumer. The API validates and publishes an event; the consumer performs the database write later. Publisher confirms, persistent messages, manual consumer acknowledgements, and idempotent inserts improve reliability. The result is a near-real-time path measured in seconds, not a hard real-time system with a guaranteed deadline.

1.3 Problem Statement and Objectives

1.3.1 Problem Statement

To design and implement a reproducible e-commerce data engineering platform that converts imperfect transaction records into a clean analytical warehouse, calculates reusable business

metrics, compares storage formats and execution modes, communicates insights through a dashboard, and demonstrates live event ingestion using RabbitMQ.

1.3.2 Objectives

Table 1.1: Practical case study objectives and implementation evidence

Objective	Implementation evidence
Explore and clean data	Profiles, safe type conversion, standardization, deduplication, and rejected records
Design an analytical schema	`dim_date`, `dim_customer`, `dim_product`, and `fact_sales`
Automate ETL	Python CLI runs generation, loading, transformation, analysis, and evidence export
Create reusable metrics	Views for daily revenue, monthly growth, and repeat-customer percentage
Demonstrate OLAP	`GROUPING SETS`, `ROLLUP`, `CUBE`, `DENSE_RANK`, `LAG`, and `LEAD`
Optimize and compare	ZSTD Parquet, partitions, indexes, thread settings, plans, and repeated timings
Present insights	Streamlit dashboard with KPI cards and decision-oriented charts
Support event ingestion	FastAPI, RabbitMQ, consumer, idempotent DuckDB storage, and five-second refresh

1.4 Scope and Report Organization

The historical dataset covers 1 January 2024 through 31 December 2025, with one order line as the business grain. The platform runs locally and can optionally be started through Docker Compose or represented through Kubernetes manifests. The live source is a simulator: the source is simulated, but the API validation, broker transport, consumer processing, database

insert, and dashboard refresh are real running components. The platform is educational and reproducible; it is not presented as a production financial or hard real-time system.

The remaining chapters review the relevant theory, compare alternative approaches, present the proposed architecture, explain the design and implementation, analyze measured results, and conclude with limitations and future scope.

CHAPTER 2

LITERATURE SURVEY

2.1 Analytical Data Management

Transactional systems are optimized for short inserts and point lookups, whereas analytical systems scan many records, select a small number of columns, and perform joins and aggregations. DuckDB follows an in-process analytical design: the engine is embedded in Python but exposes relational SQL. This avoids a separate server for a local practical while retaining repeatable, declarative transformations [1].

2.1.1 CSV and Columnar Storage

CSV represents each field as text and requires parsing during every scan. It contains little metadata and no native compression or statistics. Apache Parquet stores values by column, records metadata, and supports compression. DuckDB can push projections and filters into Parquet scans, so irrelevant columns and row groups may be skipped [2, 3]. These characteristics motivate the CSV–Parquet–DuckDB comparison in this case study.

2.1.2 Python and SQL Responsibilities

Pandas is effective for exploration, but long sequences of DataFrame operations can obscure relational intent. SQL states the desired result declaratively and is easier to audit. The implementation therefore uses Python for orchestration, API services, tests, and visualization, while DuckDB SQL performs standardization, warehouse construction, metric calculation, window analysis, and OLAP aggregation.

2.2 Dimensional Modeling and OLAP

Dimensional modeling represents a measurable business process with a central fact table connected to descriptive dimensions [4]. A fact table must declare a precise grain. Here, the grain is one order line. The date, customer, and product dimensions provide descriptive

context, while `fact_sales` stores quantity, unit price, discount, gross amount, net revenue, order status, sales channel, and shipping measures.

2.2.1 Reusable Business Metrics

A business metric should have a single documented definition. Realized revenue, for example, includes Delivered and Shipped records instead of all submitted or returned lines. Placing this expression inside a SQL view allows the dashboard and analytical queries to use the same rule. Daily revenue, monthly sales growth, and repeat-customer percentage are implemented as reusable views.

2.2.2 Window and Multidimensional Functions

Window functions calculate values across related rows without collapsing them. `LAG` compares the current month with the previous month; `LEAD` enables forward comparison; and `DENSE_RANK` orders products or categories without gaps. `GROUPING SETS`, `ROLLUP`, and `CUBE` produce detail rows, subtotals, and grand totals. These features are practical OLAP operations across time, product, geography, channel, and customer dimensions.

2.3 Event-Driven Ingestion

REST APIs provide a controlled entry point for new data. FastAPI validates incoming JSON through typed Pydantic models and returns HTTP 202 Accepted after the broker accepts the message [5]. Separating acceptance from database processing prevents the client from waiting for downstream analytical storage.

2.3.1 Message Reliability

RabbitMQ implements queues using AMQP. A durable queue survives broker restart when its messages are also persistent. Publisher confirms tell the producer that the broker accepted responsibility for the message. Consumer acknowledgements tell the broker that processing completed. These mechanisms protect different parts of the delivery path [6].

Manual acknowledgement can cause redelivery if a consumer fails after writing but before acknowledging. The consumer must therefore be idempotent. Each event has a UUID `event_id` stored as a primary key, and the insert uses `ON CONFLICT DO NOTHING`.

A repeated delivery cannot create duplicate revenue. Invalid payloads are written to an error log, while transient failures are negatively acknowledged and requeued.

2.4 Comparison of Approaches

Table 2.1: Comparison of analytical and ingestion approaches

Approach	Strength	Limitation for this case study
CSV and spreadsheet	Easy to exchange and inspect	Weak typing, repeated manual logic, and slow repeated scans
Pandas-only notebook	Flexible exploration	Procedural logic can become difficult to reuse and audit
Client-server warehouse	Strong concurrency and administration	Higher setup effort for a compact classroom practical
Spark or Hadoop	Horizontal cluster scalability	Operational overhead is unnecessary for 250,000 local rows
DuckDB, SQL, and Parquet	Embedded OLAP, direct file access, and reproducibility	Not a replacement for every high-concurrency warehouse
Direct API-to-database	Simple request path	Tight coupling and weak buffering during downstream failure
API, RabbitMQ, consumer	Decoupling, buffering, acknowledgement controls	Adds broker operation and delivery-semantics complexity

Many beginner case studies stop after loading a CSV or drawing charts. Other examples demonstrate messaging without connecting events to an analytical model. This implementation joins source capture, cleaning, rejection, dimensional modeling, reusable metrics, OLAP, storage comparison, dashboarding, event ingestion, tests, and deployment evidence in one explainable workflow.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Architecture

The proposed platform contains ingestion, processing, storage, serving, and evidence layers. Historical CSV and a public DummyJSON snapshot enter the batch layer. A Python CLI invokes DuckDB SQL to create raw, staging, rejected, dimension, fact, and view objects. The results are retained in a DuckDB database, partitioned Parquet, JSON/CSV evidence, plans, charts, and logs. Streamlit reads the analytical model for presentation.

The event layer is complementary. FastAPI validates an order and publishes it to RabbitMQ. A consumer writes accepted events to a dedicated DuckDB streaming table. The dashboard requests current metrics from the API every five seconds. The queue allows request capture and storage processing to operate independently.

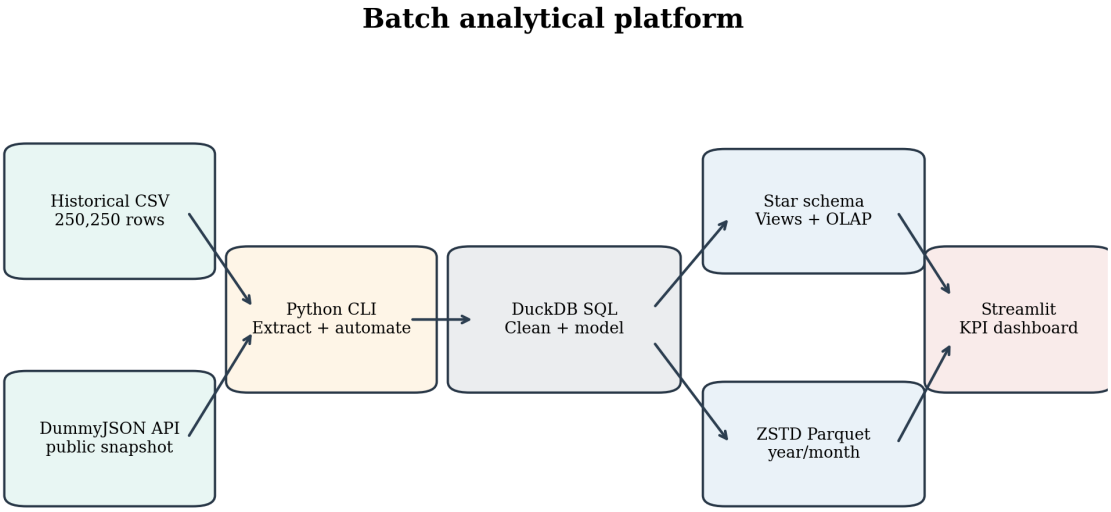


Figure 3.1: Batch analytical platform

3.1.1 Architectural Principles

- Preserve raw input before applying business rules.
- Make the grain, keys, and metric definitions explicit.

- Quarantine bad records with reasons instead of silently deleting them.
- Separate orchestration code from reusable SQL.
- Generate evidence files so measured results can be reproduced.
- Treat live delivery as at-least-once and make the database write idempotent.

3.2 System Components

Table 3.1: System components and responsibilities

Component	**Responsibility**
Generator and API snapshot	Create reproducible historical data and capture a current public sample
Python CLI	Execute pipeline stages and manage paths
DuckDB	Perform SQL transformation, modeling, analysis, and persistent storage
Parquet	Store compressed, partitioned analytical data
Streamlit	Present filters, KPIs, plots, and live orders
FastAPI	Validate JSON, publish events, and expose live metrics
RabbitMQ	Buffer order events durably between the producer and consumer
Consumer	Validate again, insert idempotently, and acknowledge processing
Docker/Kubernetes	Describe repeatable local and orchestrated deployment

3.3 Functional and Non-functional Requirements

Table 3.2: System requirements and acceptance evidence

Type	Requirement	Acceptance evidence
Functional	Load malformed CSV safely	Every field first lands as text
Functional	Clean and standardize data	Staging and rejected tables
Functional	Provide reusable metrics	Daily, monthly, and customer SQL views
Functional	Support live orders	POST endpoint, queue, consumer, and metrics endpoint
Performance	Compare storage and threads	Six repeated benchmark configurations
Reliability	Prevent duplicate events	UUID primary key and conflict handling
Maintainability	Keep modules inspectable	Separate `src`, `sql`, `tests`, and `evidence` folders
Portability	Support multiple runtimes	Windows launcher, Docker Compose, and Kubernetes manifests

3.4 Batch and Streaming Flow

The batch flow generates or selects a historical CSV, loads source fields as text, standardizes values, converts types with `TRY_CAST`, isolates invalid records, constructs dimensions and the fact table, creates views and OLAP outputs, exports Parquet, benchmarks storage, and serves the database through Streamlit.

The streaming flow is illustrated in Figure 3.2. A simulator creates approximately one order per second. FastAPI validates the order and publishes it to a durable queue. The consumer inserts the event and acknowledges only after a successful database write. The dashboard polls aggregate metrics; therefore the live count and revenue change while the stack is running.

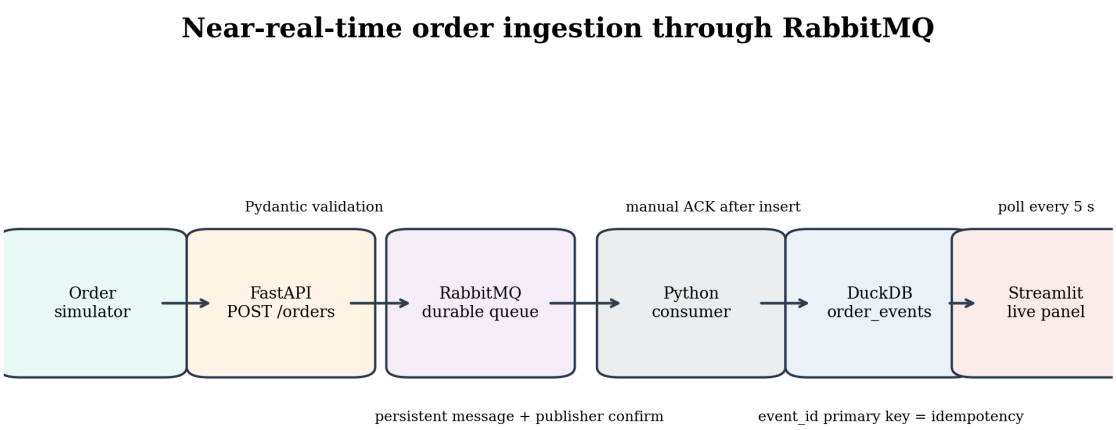


Figure 3.2: Near-real-time RabbitMQ order path

3.5 Technology Selection

DuckDB is selected for local vectorized OLAP and direct CSV/Parquet access; Parquet with ZSTD is selected for compact columnar exchange; Python is used for orchestration and application services; Streamlit provides rapid analytical storytelling; FastAPI supplies typed REST validation and OpenAPI documentation; RabbitMQ supplies durable asynchronous transport; Docker Compose represents the complete local stack; and Kubernetes manifests demonstrate deployment, persistence, probes, Services, and horizontal autoscaling.

CHAPTER 4

DESIGN AND METHODOLOGY

4.1 Data Source and Exploration

The deterministic source contains 250,250 rows, including 250 deliberately duplicated line identifiers. The business grain is one order line. Coverage is from 2024-01-01 to 2025-12-31 and includes 48 products in six categories, 37,114 unique cleaned customers, and 119,044 distinct orders in the analytical population. Defects are inserted intentionally so data-quality rules can be demonstrated and reproduced.

Exploration checks row counts, null counts, data types, distinct values, frequency distributions, numeric ranges, and business-domain consistency. Important defects include null customer identifiers, missing or non-positive prices, negative or excessive quantities, discounts outside 0–80 percent, inconsistent capitalization, malformed dates, and duplicate line identifiers. The first landing table retains fields as text so malformed numeric or date values do not abort ingestion.

Table 4.1: Source profile

Attribute	Value
Coverage	2024-01-01 through 2025-12-31
Business grain	One order line per row
Raw rows	250,250
Unique rows after deduplication	250,000
Accepted fact rows	248,555
Rejected rows	1,445
Rejection rate	0.578%
Distinct orders	119,044
Distinct cleaned customers	37,114

4.2 ETL and Data Quality Rules

The ETL follows five controlled stages: extract, raw landing, standardization, validation, and publication. The Python CLI extracts the selected source and opens DuckDB. SQL creates schemas and loads raw values. Standardization trims text, normalizes case, and uses `TRY_CAST`; invalid conversions become NULL and can be handled by explicit rules. Duplicate rows are ranked by `ROW_NUMBER` over `line_id`, and one deterministic record is kept.

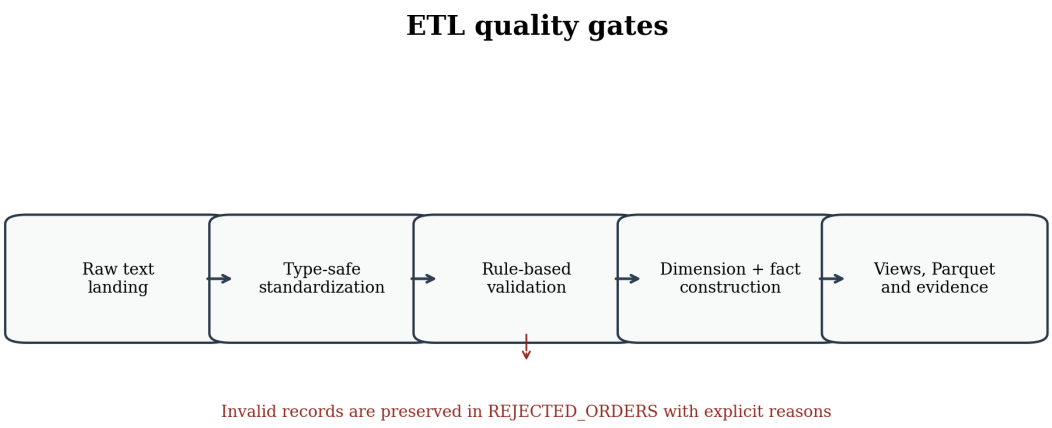


Figure 4.1: ETL quality gates and rejected-record side output

Table 4.2: Cleaning and validation rules

Field or issue	Rule	Treatment
Duplicate `line_id`	Rank by `ROW_NUMBER`	Keep one deterministic row
Dates and numerics	Convert with `TRY_CAST`	Invalid conversion becomes NULL
Customer/product IDs	Required	Reject the row when missing
Quantity	1 to 20	Reject outside the range
Unit price	Greater than zero	Reject missing or non-positive values
Discount	0 to 80 percent	Reject outside the range
Category, region, status	Trim and normalize case	Map to controlled values

Every rejected record retains its original values and a semicolon-separated reason for rejection. A quality JSON file records the raw, deduplicated, accepted, and rejected counts plus file sizes and runtime. This design makes the cleaning result auditable: 250,000 = 248,555 + 1,445.

4.3 Relational and Dimensional Model

The source can first be viewed as normalized operational entities: Customer, Product, Order, and Order Line. In the analytical model, descriptive attributes are denormalized into

dimensions and measures are stored in a fact table. This is a deliberate warehouse design choice: it minimizes join depth for analytical queries while retaining stable surrogate keys.

Analytical star schema and quality quarantine

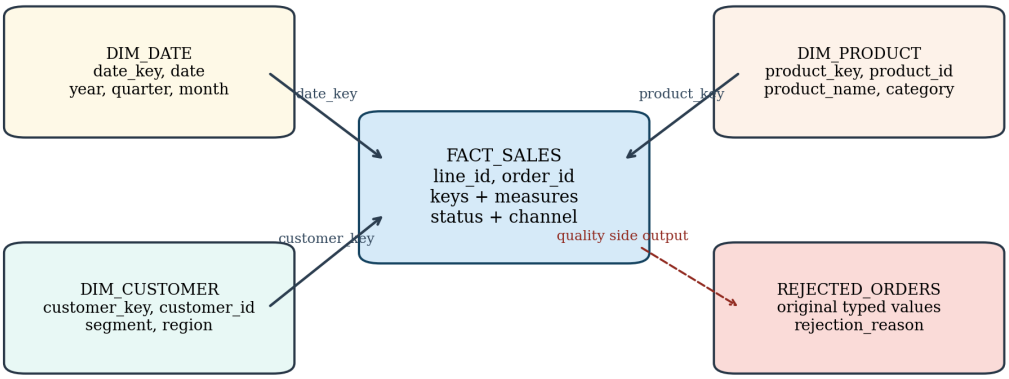


Figure 4.2: Analytical star schema and quality quarantine

Table 4.3: Warehouse tables and grain

Table	Grain/key	Purpose
`dim_date`	One calendar date	Year, quarter, month, and weekday attributes
`dim_customer`	One customer	Customer identifier, segment, and region
`dim_product`	One product	Product identifier, name, and category
`fact_sales`	One accepted order line	Foreign keys, quantity, price, discount, revenue, status, and channel
`rejected_orders`	One invalid source line	Original values and rejection reason
`streaming. order_events`	One live event UUID	Validated near-real-time orders

Surrogate integer keys stabilize joins even if natural identifiers change. The fact table keeps `order_id` and `line_id` as degenerate dimensions for traceability. The primary key on `line_id` prevents duplicate fact records. Indexes document selective access paths for date, customer, and product keys, although full analytical scans may not always benefit from an index.

4.4 Reusable SQL Views and Window Functions

The enriched sales view joins the fact table to all dimensions once. The daily revenue view groups realized revenue, orders, and customers by date. The monthly growth view uses `LAG` to compare each month with its predecessor. The repeat-customer view identifies customers with more than one fulfilled order and divides them by all fulfilled customers.

Listing 4.1: Representative monthly growth view

```
WITH monthly_sales AS (
  SELECT date_trunc('month', order_date) AS month,
         SUM(net_revenue) AS revenue
  FROM warehouse.vw_sales_enriched
  WHERE order_status IN ('Delivered', 'Shipped')
  GROUP BY 1
), compared AS (
  SELECT month, revenue,
         LAG(revenue) OVER (ORDER BY month) AS previous_revenue
  FROM monthly_sales
)
SELECT month, revenue, previous_revenue,
       ROUND(100.0 * (revenue - previous_revenue)
             / NULLIF(previous_revenue, 0), 2) AS growth_pct
FROM compared
ORDER BY month;
```

For ranking, `DENSE_RANK` operates within a region so ties share a rank without producing gaps. `LEAD` supports a forward month comparison. These functions preserve row detail while adding analytical context.

4.5 OLAP and Query Optimization

`ROLLUP(region, category)` returns region-category details, regional subtotals, and an overall total. `CUBE(region, sales_channel)` returns all combinations of region and channel, including one-dimensional totals. `GROUPING SETS` requests only the levels needed by the dashboard and therefore avoids unnecessary combinations.

Optimization is evaluated through design and measurement. The pipeline writes Parquet with ZSTD compression and year/month folder partitions. DuckDB reads only required columns and can prune partitions when filters refer to partition keys. The database persists cleaned tables so repeated queries avoid reparsing CSV. Thread settings compare local parallel execution, and `EXPLAIN ANALYZE` records actual operators and timing. One-thread versus four-thread measurements are local execution modes, not a claim of multi-node cluster performance.

4.6 RabbitMQ Live Pipeline

FastAPI validates required identifiers, enumerated fields, date/time values, positive prices and quantities, and the allowed discount range. It creates a UUID when an event identifier is not supplied. The publisher declares a durable queue, marks the message persistent, and enables publisher confirms before returning HTTP 202.

The consumer uses bounded prefetch and validates the message again. It inserts into `streaming.order_events` and acknowledges only after the transaction completes. Duplicate UUIDs are ignored by the primary key. Invalid data is logged without requeue; transient failures are negatively acknowledged with requeue. This combination implements at-least-once transport with exactly-once effect for the protected insert.

The dashboard requests `/metrics` and `/orders/recent` every five seconds. The part that is live is the sequence from a newly generated or posted order through the API, broker, consumer, database, and refreshed metric cards. Historical charts remain batch analytics.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Data Quality Results

The full evidence run read 250,250 source rows. Deduplication retained 250,000 unique line identifiers. Business and type rules accepted 248,555 fact rows and preserved 1,445 records in the rejected table, giving a rejection rate of 0.578 percent. No row disappears without reconciliation. The complete batch pipeline finished in 7.061 seconds on the validation machine.

The DuckDB file measured 22.012 MB and the partitioned Parquet output measured 6.666 MB. These values are not directly equivalent because the database stores multiple tables, indexes, views, and metadata, whereas the Parquet measurement covers the exported analytical dataset. Nevertheless, the result demonstrates the compactness of compressed columnar storage.

Table 5.1: Data-quality and storage results

Measure	**Observed value**
Raw input rows	250,250
Rows after duplicate removal	250,000
Accepted fact rows	248,555
Rejected rows retained with reasons	1,445
Rejection rate	0.578%
DuckDB database size	22.012 MB
Partitioned ZSTD Parquet size	6.666 MB
Pipeline elapsed time	7.061 seconds

5.2 Business KPIs

Table 5.2: Business KPI summary

KPI	Measured result	Meaning
Realized revenue	$28,777,038.79	Delivered and shipped line revenue after discount
Distinct orders	119,044	Unique orders in the clean analytical population
Distinct customers	37,114	Customers represented in the warehouse
Average fulfilled line	$131.59	Average realized value per included line
Return rate	7.11%	Share of order lines marked Returned
Repeat-customer percentage	77.51%	Customers with more than one realized order
Top category	Electronics	Highest realized category revenue
Top region	South	Highest realized regional revenue

Revenue reached approximately $28.78 million across 119,044 orders. The 77.51 percent repeat-customer measure suggests strong recurring purchase behavior in the generated historical population. The 7.11 percent return rate identifies a separate quality-of-sale concern. These findings are illustrative because the large historical source is synthetic; the value of the case study lies in the method, reproducibility, and traceability of the measures.

5.3 Category, Region, and Time Analysis

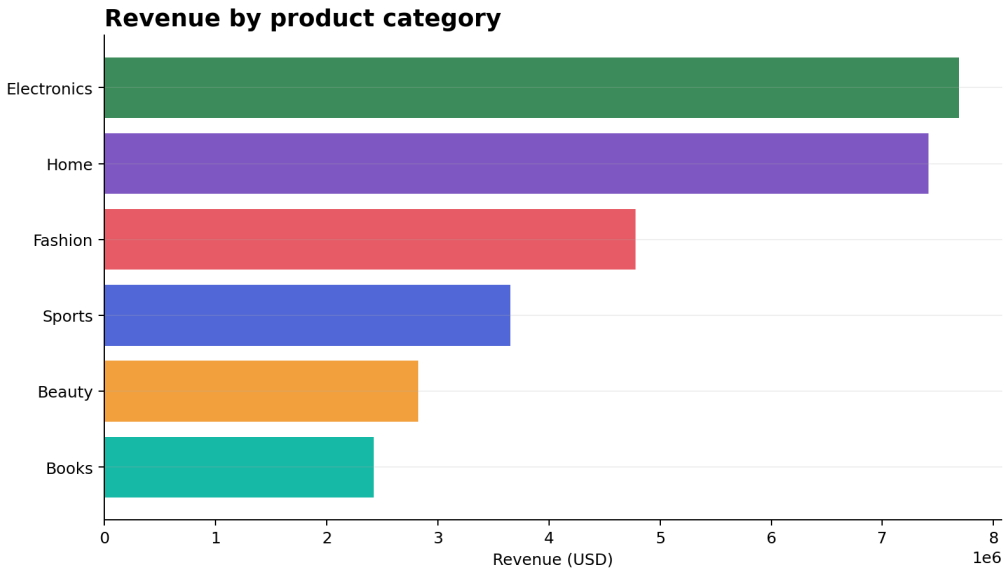
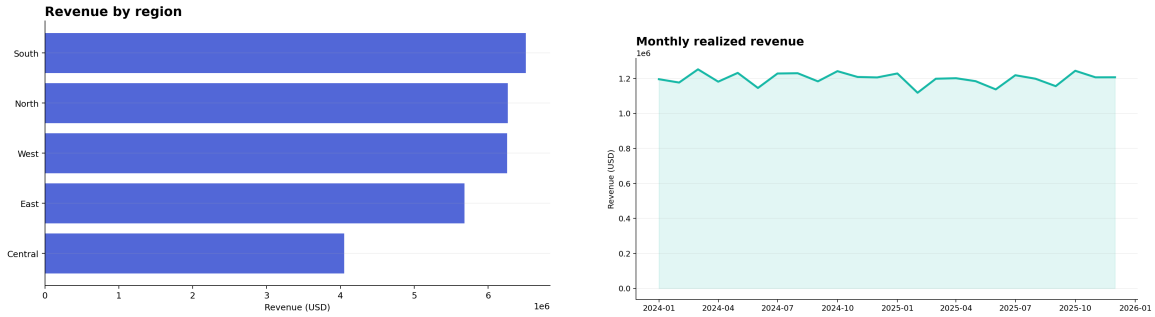


Figure 5.1: Realized revenue by product category

Electronics produces about $7.69 million and leads realized revenue. This is a revenue conclusion, not a profit conclusion, because product cost is not present in the source. The result can guide inventory or promotion review, while margin analysis would require cost and procurement data.



(a) Revenue by region

(b) Monthly revenue trend

Figure 5.2: Regional and time-based analysis

The South region records the highest realized revenue at approximately $6.51 million. Because region is stored once in `dim_customer`, ranking, filtering, and OLAP use the same controlled definition. Monthly revenue remains fairly stable over the two-year period with visible month-to-month variation. The monthly-growth view calculates the exact change with `LAG` and guards against division by zero.

5.4 Performance Evaluation

Each benchmark configuration is warmed and then timed five times. The report uses the median as the primary statistic and also records minimum and maximum times. The same aggregation is executed against CSV, Parquet, and a persistent DuckDB table under one-thread and four-thread settings.

Table 5.3: Storage-format and thread benchmark

Storage	Threads	Median ms	Min ms	Max ms
CSV	1	674.031	665.540	688.396
Parquet	1	104.650	100.067	114.811
DuckDB	1	44.775	42.669	48.432
CSV	4	330.050	323.678	333.697
Parquet	4	57.009	49.927	60.355
DuckDB	4	29.319	27.484	30.068

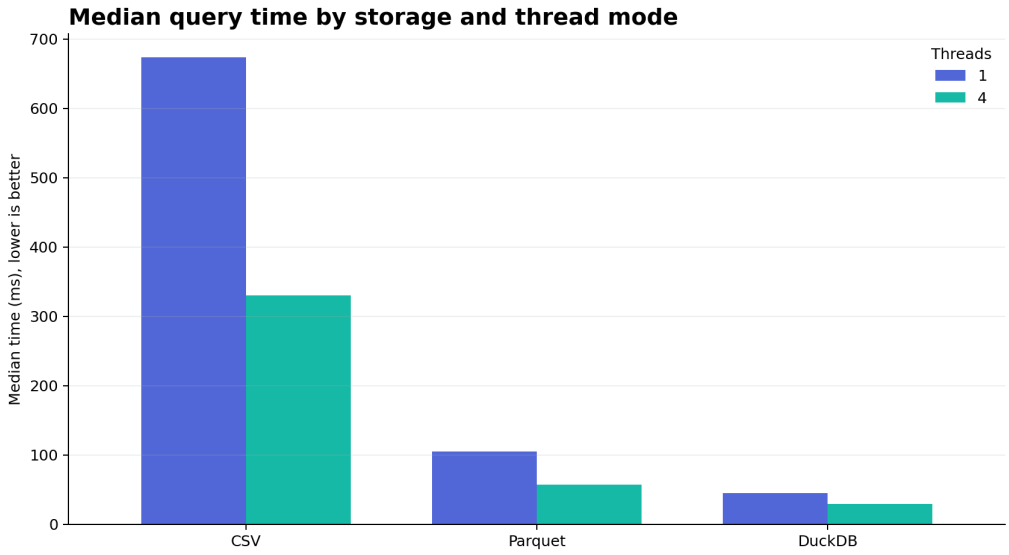


Figure 5.3: Median query time by storage and thread mode

The median falls from 674.031 ms for single-thread CSV to 29.319 ms for four-thread native DuckDB, a 22.99-times difference for this query, machine, dataset, and warmed procedure. Four threads improve CSV by 2.04 times, Parquet by 1.84 times, and DuckDB by 1.53 times. Scaling is not perfectly linear because parsing, I/O, scheduling, row-group

layout, and aggregation have different costs. These figures are bounded experimental results, not universal database claims.

5.5 Dashboard and Business Storytelling

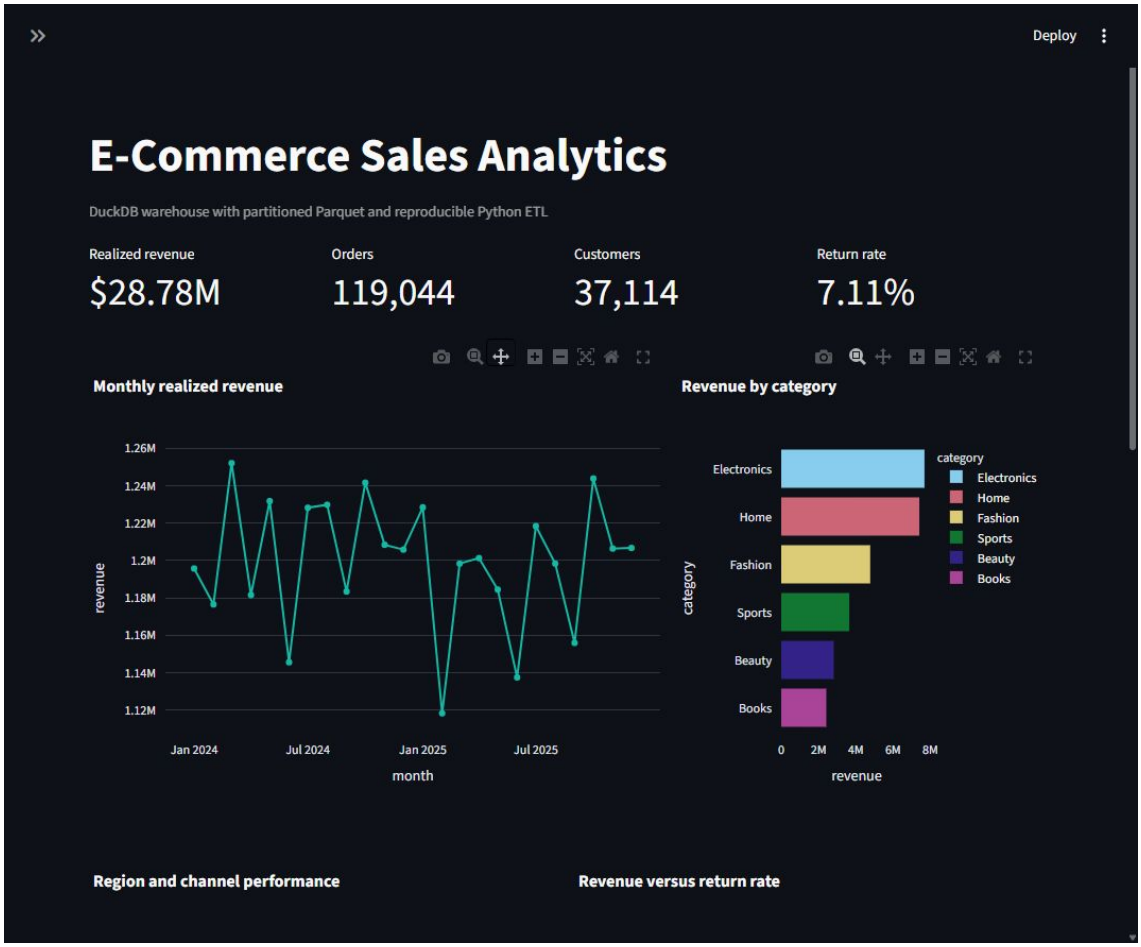


Figure 5.4: Streamlit analytical dashboard

The dashboard presents headline KPIs first so the viewer understands overall scale before exploring categories, regions, time, channels, and return behavior. Filters query the warehouse rather than independent extracts. The interface answers the following questions: What is current realized revenue? Which categories and regions lead? How does revenue change over time? What percentage of customers return? Which areas combine high sales with return risk? Are live order counts and revenue changing as new events are consumed?

5.6 Live Pipeline Validation

Automated checks cover the historical CSV-to-DuckDB pipeline, event idempotency, invalid payload rejection, and durable-queue/persistent-message settings. Docker Compose and Kubernetes Kustomize files are structurally validated. The live design uses a single writer for the DuckDB event file; the dashboard reads live metrics through FastAPI instead of independently opening a competing writable connection.

Table 5.4: RabbitMQ live-path validation

Stage	**Implementation**	**Reliability behavior**
Capture	FastAPI `POST /orders`	Typed validation and generated event UUID
Publish	RabbitMQ durable queue	Persistent message and publisher confirmation
Consume	Python Pika consumer	Bounded prefetch and second validation
Store	DuckDB event table	Primary-key idempotency and conflict ignore
Complete	Manual acknowledgement	ACK only after successful insert
Present	API metrics and Streamlit	Five-second polling of processed events

On the earlier validation host, the complete RabbitMQ container run could not finish because the Docker Desktop engine socket was unavailable. The case study therefore distinguishes implemented and tested delivery logic from a specific host-runtime limitation. When Docker Desktop is available, the supplied Compose stack starts RabbitMQ, the API/consumer service, simulator, and dashboard.

5.7 Requirement Traceability and Bonus Work

Table 5.5: Requirement-to-evidence traceability

Requirement	**Evidence**
Load CSV/Parquet and perform SQL	CLI pipeline, DuckDB warehouse, Parquet export, and SQL files
Joins and aggregations	Star-schema joins, KPI views, window functions, and OLAP queries
Optimize performance	ZSTD Parquet, partitions, indexes, threads, benchmarks, and plans
Capture outputs	JSON/CSV evidence, charts, dashboard screenshot, logs, and test report
Use a real API	DummyJSON snapshot: 194 products, 208 carts, and 800 flattened cart lines
Implement Kubernetes	Namespace, Deployments, Services, Secret, PVC, probes, and HPA
Compare execution modes	One-thread versus four-thread local analytical execution
Support live streaming	FastAPI → RabbitMQ → consumer → DuckDB → dashboard

The public DummyJSON snapshot gives the repository an externally observed data sample. The historical dataset remains deterministic and synthetic so all defects and performance results can be reproduced. Kubernetes resources show deployment design; they do not by themselves prove a production multi-node benchmark. This distinction prevents bonus features from being overstated.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The practical case study demonstrates a complete and explainable data engineering life cycle for e-commerce analytics. It begins with raw files and a public API snapshot, profiles data quality, preserves raw values, standardizes and validates records, quarantines failures, constructs a dimensional model, defines reusable metrics, performs window and multidimensional analysis, exports compressed columnar data, measures query performance, and presents decision-oriented findings through an interactive dashboard.

The implementation processes 250,250 raw rows into 248,555 accepted fact rows while preserving 1,445 rejected records with reasons. The warehouse reports $28.777 million in realized revenue, 119,044 orders, and a 77.51 percent repeat-customer rate. For the selected query, the benchmark records a 22.99-times median difference between single-thread CSV and four-thread native DuckDB.

The RabbitMQ extension shows how data can enter continuously. Typed API validation, durable queuing, persistent delivery, publisher confirmation, manual acknowledgement, idempotent insertion, and dashboard polling are represented in the implementation. The correct claim is near-real-time event processing; it is not strict hard real time.

6.2 Limitations

- The large historical dataset is synthetic, although the public API snapshot is externally observed.
- Cost, profit, inventory, marketing spend, and customer-consent fields are outside the source model.
- The benchmark measures one analytical query on one host and is not a universal database comparison.
- The dashboard polls for updates instead of using a server-push protocol.

- DuckDB is a single-node embedded warehouse; high-concurrency multi-writer production behavior is not evaluated.
- A prior validation host had an unavailable Docker engine, so runtime evidence is separated from code and configuration validation.

6.3 Future Scope

- Replace the simulator with checkout, payment, shipment, and return event sources.
- Add explicit event schema versions and compatibility rules.
- Add retry queues, a dead-letter exchange, queue-depth monitoring, alerts, and end-to-end latency metrics.
- Use PostgreSQL or a cloud warehouse for concurrent serving while retaining DuckDB for local analytics.
- Schedule incremental models with Airflow or Prefect and publish expectation-suite validation reports.
- Add role-based access, managed secrets, encryption, retention policies, and personally identifiable information controls.
- Run Kubernetes load tests across replicas and measure throughput, recovery time, autoscaling behavior, and cost.

6.4 Learning Outcomes

The practical shows that data quality must be measurable and auditable; a clear grain and reusable metric layer prevent contradictory analysis; storage and execution choices should be supported by repeatable measurements; reliable messaging requires producer confirmation, consumer acknowledgement, and idempotent storage together; and a dashboard is the final communication layer, not a substitute for correct upstream engineering.

Bibliography

[1] M. Raasveldt and H. Mühleisen, "DuckDB: an Embeddable Analytical Database," *Proceedings of the 2019 International Conference on Management of Data*, pp. 1981–1984, 2019.

[2] DuckDB Foundation, "Reading and Writing Parquet Files," DuckDB Documentation. https://duckdb.org/docs/stable/data/parquet/overview

[3] Apache Software Foundation, "Apache Parquet Documentation." https://parquet.apache.org/docs/

[4] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed., Wiley, 2013.

[5] FastAPI, "Request Body," FastAPI Documentation. https://fastapi.tiangolo.com/tutorial/body/

[6] RabbitMQ Team, "Consumer Acknowledgements and Publisher Confirms." https://www.rabbitmq.com/docs/confirms

[7] DuckDB Foundation, "Window Functions." https://duckdb.org/docs/stable/sql/functions/window_functions

[8] DuckDB Foundation, "GROUPING SETS." https://duckdb.org/docs/stable/sql/query_syntax/grouping_sets

[9] Streamlit, "Streamlit Documentation." https://docs.streamlit.io/

[10] Kubernetes Authors, "Horizontal Pod Autoscaling." https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/

[11] DummyJSON, "Products and Carts API Documentation." https://dummyjson.com/docs

CHAPTER A

EXECUTION GUIDE

A.1 One-Click Windows Start

Run `Start_ECommerce_From_Desktop.cmd` from the repository folder or use the Desktop launcher. The PowerShell module verifies Python, prepares the local environment, generates the warehouse if required, starts Streamlit, and opens `http://127.0.0.1:8501/`.

A.2 Batch Commands

```
python -m src.cli all --rows 250000 --runs 5
streamlit run src/dashboard.py
```

A.3 Full RabbitMQ Stack

```
docker compose up --build
docker compose --profile demo up --build
```

The main endpoints are:

- Streamlit dashboard: `http://localhost:8501`
- FastAPI documentation: `http://localhost:8001/docs`
- RabbitMQ management: `http://localhost:15672`

A.4 Suggested Demonstration Sequence

1. Open the dashboard and explain the historical KPI cards and charts.
2. Show raw, rejected, dimension, fact, and view objects in DuckDB.
3. Run one window-function query and one grouping-set query.
4. Explain the benchmark procedure, evidence, and bounded interpretation.
5. Start the live stack and open the RabbitMQ management page.
6. Post or simulate orders and show queue activity plus the changing live dashboard panel.

CHAPTER B

TEAM CONTRIBUTION

Table B.1: Primary responsibilities of team members

Team member	**Roll number**	**Primary responsibility**
Harsh Birje	225A8127	Problem statement, EDA, data quality, batch ingestion, and introduction
Aayush Chavanke	225A8128	Star schema, SQL views, window functions, OLAP, and normalization
Aniket Nikam	225A8135	Parquet optimization, benchmarks, dashboard, evidence, and repository coordination
Lokesh Ture	225A8138	FastAPI, RabbitMQ producer/consumer, reliability, Docker, and Kubernetes

All four members are expected to understand the complete practical case study and answer questions from any chapter. The distribution assigns primary ownership for preparation and presentation; it does not limit technical understanding.